- Token attributes can be moved between the JWT and the database depending on how sensitive they are or how likely they are to change. You might want to store some basic information about the user in the JWT but store a last activity time for implementing *idle timeouts* in the database as it will change frequently.

  DEFINITION An ***idle timeout*** (or *inactivity logout*) automatically revokes an authentication token if it hasn't been used for a certain amount of time. This can be used to automatically log out a user if they have stopped using your API but have forgotten to log out manually. Idle timeouts are a good way to balance usability with security as the overall token expiry time can be left long, ensuring that the user isn't prompted to reauthenticate too often, while ensuring that tokens are invalidated quickly when they are no longer being used. You can implement an idle timeout by updating a last-used attribute on the token every time it is read.

Listing 6.11 shows the `JwtTokenStore` updated to whitelist tokens in the database. It does this by taking an instance of the `DatabaseTokenStore` as a constructor argument and uses that to create a dummy token with no attributes. If you wanted to move attributes from the JWT to the database, you can do that here by populating the attributes in the database token and removing them from the JWT token. The token ID returned from the database is then stored inside the JWT as the standard JWT ID (`jti`) claim. Open JwtTokenStore.java in your editor and update it to whitelist tokens in the database as in the listing.